

Machine Learning

Probability & Estimation

Bayes' Theorem (how to update beliefs given evidence):
 $P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$; denominator:
 $P(X) = \sum_k P(X|Y_k)P(Y_k)$
Independent: $P(X, Y) = P(X)P(Y)$; uncorrelated:
 $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$

MLE (pick θ that makes data most likely):

$\hat{\theta} = \arg \max_{\theta} \sum_i \log p(x_i|\theta)$. Gaussian: $\hat{\mu} = \bar{x}$,

$\hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \bar{x})^2$

MAP (MLE + prior belief about θ):

$\hat{\theta} = \arg \max [\log p(\text{data}|\theta) + \log p(\theta)]$

Ridge $\equiv$ Gaussian prior; LASSO $\equiv$ Laplace prior (connection to regularization!).

$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$;

$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$

Bias-Variance Tradeoff

Expected Test MSE =

$$\underbrace{\mathbb{E}[\hat{f}] - f}_{\text{Bias}^2}^2 + \underbrace{\mathbb{E}[(\hat{f} - \mathbb{E}[\hat{f}])^2]}_{\text{Variance}} + \underbrace{\sigma_{\epsilon}^2}_{\text{irreducible}}$$

Bias = error from wrong model assumptions (too simple $\rightarrow$ underfits).

Variance = sensitivity to which training data you happen to get (too complex $\rightarrow$ overfits).

Irreducible noise σ_{ϵ}^2 : cannot be reduced no matter the model.

Model complexity $\uparrow \Rightarrow$ bias $\downarrow$, var $\uparrow$. Regularization $\Rightarrow$ bias $\uparrow$, var $\downarrow$.

Test error forms a U-shape vs. complexity; optimal = minimum of that U.

When to Use (and Not Use) ML

Use ML when: pattern too complex for hand-coded rules; large labeled data available; performance improves with more data; outcome is measurable.

Don't use ML when: a simple rule-based system works; data is too scarce; full explainability required (legal/medical); problem not well-defined; cost of errors is too high to accept a black box.

Before modeling: define target variable; assess data quantity/quality; establish a simple baseline; clarify error costs.

KNN (K-Nearest Neighbors)

Predict by averaging (regression) or majority vote (classification) over the k closest training points.

Non-parametric: no model is learned, just memorizes data.

Use when: Nonlinear boundaries; small datasets;

interpretable local predictions.

Bias $\leftrightarrow$ Var: $k \downarrow$: each point gets its own neighborhood $\rightarrow$ low bias, high var (wiggly, overfits). $k \uparrow$: average over large region $\rightarrow$ high bias, low var (over-smoothed). $k = 1$ perfectly fits training set.

Key HPs: k (tune by CV, typical 3-20); distance metric (Euclidean default; scale features first!).

Linear Regression

Models y as a linear function of inputs: $\hat{y} = \mathbf{X}\beta$;

minimizes RSS = $\|\mathbf{y} - \mathbf{X}\beta\|^2$.

OLS closed form: $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ (requires $\mathbf{X}^T \mathbf{X}$ invertible).

Gauss-Markov: OLS is BLUE when $\mathbb{E}[\epsilon] = 0$,

$\text{Var}(\epsilon) = \sigma^2 \mathbf{I}$.

GD alternative: $\beta \leftarrow \beta - \eta \nabla_{\beta} \text{RSS}$; $\nabla = 2\mathbf{X}^T (\mathbf{X}\beta - \mathbf{y})$.

Nonlinear features: replace $\mathbf{x}$ with $\phi(\mathbf{x}) = [1, x, x^2, \dots]$; same OLS formula applies.

$R^2 = 1 - SS_{res}/SS_{tot}$; adjusted R^2 penalizes extra predictors. High VIF (>10) signals collinearity.

Feature selection: *forward* starts empty and adds best predictor each step; *backward* starts full and removes least useful predictor each step.

Norms / projection: $\|\mathbf{x}\|_1 = \sum_i |x_i|$, $\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$,

$\text{proj}_{\mathbf{y}}(\mathbf{x}) = \frac{\mathbf{x}^T \mathbf{y}}{\mathbf{y}^T \mathbf{y}} \mathbf{y}$.

Use when: Continuous outcome; interpretable coefficients;

roughly linear relationship.

Bias $\leftrightarrow$ Var: *OLS: unregularized $\rightarrow$ zero bias (if model correct) but high var when p large. Adding polynomial features: bias $\downarrow$, var $\uparrow$.*

Key HPs: Polynomial degree d (bias $\downarrow$ /var $\uparrow$ as $d \uparrow$); regularization λ ; learning rate η for GD.

Assumptions (LINE): **Linearity** $\mathbb{E}[Y | X] = X\beta$ (linear in parameters); **Independence** observations/errors independent; **Normality** errors approx. normal (mainly for CI/tests); **Equal variance** $\text{Var}(\epsilon_i | X) = \sigma^2$ (homoscedasticity).

Heteroscedasticity: non-constant error variance; residual spread changes with fitted values $\Rightarrow$ OLS still unbiased but inefficient, SEs/p-values unreliable; fix with transform, better features/interactions, or robust SEs.

Multicollinearity: predictors highly correlated $\Rightarrow$ unstable coefficients, large SEs, sign flips possible; check VIF; ridge helps.

Leverage: unusual x far from predictor center; hat value $h_{ii} = x_i^T (\mathbf{X}^T \mathbf{X})^{-1} x_i$; high leverage means strong potential to affect fit.

Influential point: materially changes fitted model, often high leverage + large residual; check Cook's distance.

Outlier: unusual y given x (large residual). **Key distinction:** outlier = unusual response; leverage = unusual predictors; influential = changes fit a lot.

Diagnostics: residuals vs fitted $\rightarrow$ nonlinearity/heteroscedasticity; Q-Q plot $\rightarrow$ normality; good residual plot looks random/patternless.

Logistic Regression & Perceptron

Logistic regression models $P(Y = 1|\mathbf{x})$ directly (discriminative). Models log-odds as linear:

$\log \frac{1}{1-p} = \mathbf{x}^T \beta \Rightarrow P(Y=1|\mathbf{x}) = \sigma(\mathbf{x}^T \beta) =$

$\frac{1}{1 + e^{-\mathbf{x}^T \beta}} \in (0, 1)$

Cross-entropy loss:

$\mathcal{L} = -\frac{1}{n} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$

No closed form; optimize by GD. Gradient:

$\nabla_{\beta} \mathcal{L} = \frac{1}{n} \mathbf{X}^T (\hat{\mathbf{p}} - \mathbf{y})$

Softmax for multiclass: $P(Y=k|\mathbf{x}) = \frac{e^{\mathbf{x}^T \beta_k}}{\sum_j e^{\mathbf{x}^T \beta_j}}$

Perceptron: hard threshold classifier $\hat{y} = \text{sign}(\theta \cdot \mathbf{x} + \theta_0)$. Update rule: $\theta \leftarrow \theta + y_i \mathbf{x}_i$ whenever a point is misclassified.

Converges in $\leq R^2/\gamma^2$ steps only if data are linearly separable (R =radius, γ =margin).

Use when: Binary/multiclass classification; calibrated probabilities needed; linear decision boundary is sufficient.

Bias $\leftrightarrow$ Var: Linear boundary $\rightarrow$ high bias if true boundary is curved. Low var with regularization. Adding interaction/poly features lowers bias.

Key HPs: Regularization $C = 1/\lambda$: $C \downarrow$ (more reg) $\rightarrow$ bias $\uparrow$, var $\downarrow$; $C \uparrow \rightarrow$ overfit risk. class.weight for imbalance.

Performance Evaluation

	Pred +	Pred -
Act +	TP	FN
Act -	FP	TN

TPR (Recall) = $\frac{TP}{TP+FN}$; FPR = $\frac{FP}{FP+TN}$;

Precision = $\frac{TP}{TP+FP}$; Acc = $\frac{TP+TN}{N}$

$F_1 = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$; $F_{\beta} > 1$ weights recall more; $\beta < 1$ weights precision more.

Computing metrics from score table: given a list of samples with true labels and model confidence scores, sort by score descending. For a threshold τ : predict + if score $\geq \tau$, else -. Count TP/FP/FN/TN, then apply formulas above. Sweeping τ from high $\rightarrow$ low traces the ROC curve (first point: all -; last point: all +).

ROC curve: plot TPR vs FPR as τ varies.

AUC $\in$ [0.5, 1]; AUC= 0.5 = random guess. Raising $\tau \Rightarrow$ fewer positives $\Rightarrow$ precision $\uparrow$, recall $\downarrow$. **PR curve** better for imbalanced data (minority class matters more).

ROC operating point — expected cost procedure:

Given costs c_{FP} , c_{FN} , c_{TP} , c_{TN} and class prevalence $p_+ = P(Y = +)$:

Expected cost at threshold τ :

Cost(τ) = $c_{FP} \cdot \text{FPR}(\tau) \cdot (1 - p_+) + c_{FN} \cdot (1 - \text{TPR}(\tau)) \cdot p_+ - c_{TP} \cdot \text{TPR}(\tau) \cdot p_+ - c_{TN} \cdot \text{TNR}(\tau) \cdot (1 - p_+)$

In practice: evaluate cost at each candidate τ (each ROC point); pick $\tau^* = \arg \min \text{Cost}(\tau)$.

Shortcut (simplified): optimal

$\tau^* = \frac{c_{FP}(1-p_+)}{c_{FP}(1-p_+) + c_{FN} \cdot p_+}$ with 0-1 payoffs on TP/TN.

High c_{FN} (miss cancer) $\rightarrow$ lower τ (catch more positives, accept more FP). High c_{FP} $\rightarrow$ raise τ . Class prevalence $p_+ \downarrow$ (rare disease) $\rightarrow$ also shifts optimal τ upward (harder to justify alarming).

$\bar{R}^2 = 1 - SS_{res}/SS_{tot} \in (-\infty, 1]$ (1=perfect; 0=same as predicting mean). MSE = $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$;

MAE = $\frac{1}{n} \sum |y_i - \hat{y}_i|$.

Class imbalance: accuracy misleading; prefer F_1 /AUC/PR curve. Fixes: SMOTE (oversample minority), undersample majority, adjust threshold τ , use class_weight.

Model Selection & Cross-Validation

k-fold CV: split data into k equal folds; train on $k - 1$, test on remaining 1; rotate and average. $k=5$ or 10 is standard. LOOCV ($k=n$): unbiased but high variance and expensive.

Stratified CV: preserve class proportions in each fold (especially important for imbalanced data).

Experimental design — two distinct goals:

Performance evaluation (how good is my model?): use held-out test set or CV; report point estimate + confidence interval. Never tune on test set.

Performance comparison (is model A better than B?): use paired tests (same folds for both models) so differences in folds cancel out. A result is "really better" only if the difference exceeds chance variation.

Interpreting under uncertainty: report mean $\pm$ std (or CI) of CV scores. Overlapping CIs does NOT mean models are equal — use paired t-test or McNemar's test.

1-SE rule: prefer simpler model if it's within 1 standard error of the best CV score.

Train/Val/Test split: train to fit; val to tune HPs; test only once at the end. **Data leakage:** fit all preprocessing (scaling, imputation) on training fold only — if you use full-dataset stats before splitting, val/test estimates are optimistically biased.

Key HPs: Grid search (exhaustive); random search (efficient, often just as good); Bayesian optimization (sequential, best for expensive models).

Decision Theory

Loss $\lambda(\alpha_i|\omega_j)$ = cost of predicting i when truth is j ; **risk** $R(\alpha_i|x) = \sum_j \lambda(\alpha_i|\omega_j)P(\omega_j|x)$. Pick action with minimum expected risk.

	$\hat{Y}=1$	$\hat{Y}=0$
$\bar{Y}=1$	λ_{11}	λ_{01}
$\bar{Y}=0$	λ_{10}	λ_{00}

$R(\tau) = \sum_{i,j} \lambda_{ij} P(\hat{Y}=i|Y=j, \tau) P(Y=j)$

If only mistakes cost: $R(\tau) = \lambda_{10} P_{FP}(\tau) + \lambda_{01} P_{FN}(\tau) = \lambda_{FP} \text{FPR} + \lambda_{FN}(1 - \text{TPR})$.

With 0-1 loss, choose most probable class:

$f^*(x) = \arg \max_k P(Y=k|x)$ (Bayes classifier), with Bayes risk $R^* = \inf_f R(f)$.

ROC operating point: high $c_{FN} \Rightarrow$ lower τ (more sensitive); high $c_{FP} \Rightarrow$ raise τ (more specific).

Neyman-Pearson: maximize TPR subject to $\text{FPR} \leq \alpha$. Excess risk:

$R(f) - R^* = \mathbb{E}[|2P(Y=1|X) - 1|1[f(x) \neq f^*(x)]]$.

Regularization

Regularization penalizes model complexity to reduce variance (overfitting) at the cost of some bias.

Ridge (L2): $\hat{\beta} = \arg \min \|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda \|\beta\|^2$. Closed form: $\hat{\beta}^{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$ (always invertible). Shrinks all coefficients toward zero; none become exactly zero. Best when many features all contribute a little.

LASSO (L1): $\hat{\beta} = \arg \min \|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda \sum_j |\beta_j|$.

Produces sparse solutions (many $\beta_j=0$) $\Rightarrow$ automatic feature selection. No closed form; use coordinate descent.

Elastic Net: $\lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2$. Combines sparsity (LASSO) and grouping effect (Ridge selects correlated features together).

Bias $\leftrightarrow$ Var: $\lambda \uparrow$: more shrinkage $\rightarrow$ bias $\uparrow$, var $\downarrow$. $\lambda = 0$: OLS (high var). $\lambda \rightarrow \infty$: $\hat{\beta} \rightarrow \mathbf{0}$ (underfit).

Key HPs: λ : search log-scale (e.g. 10^{-4} to 10^2) by CV. Elastic net ratio $\rho \in [0, 1]$ (0=Ridge, 1=LASSO).

Other regularizers: early stopping (halt GD before convergence $\equiv$ implicit L2); dropout (randomly zero

activations during training to prevent co-adaptation); data augmentation (more training variety); AIC/BIC for model comparison (BIC penalizes more: $k \log n - 2\ell$).

Generative vs. Discriminative Models

Discriminative (e.g. logistic reg): model $P(Y|X)$ directly. Better with lots of data; don't make distribution assumptions.

Generative (e.g. NB, LDA): model $P(X|Y)$ and $P(Y)$; recover $P(Y|X) \propto P(X|Y)P(Y)$ via Bayes. Better with less data; can generate new samples; make stronger assumptions.

Naïve Bayes: assumes features are conditionally independent given class.

$\hat{y} = \arg \max_k [\log P(Y=k) + \sum_j \log P(x_j|Y=k)]$

Gaussian NB: $P(x_j|Y=k) = \mathcal{N}(\mu_{jk}, \sigma_{jk}^2)$; Bernoulli NB: binary features; Multinomial NB: word counts (text).

Use when: Text classification, spam filtering, fast baseline, high-dimensional discrete features, small n .

Bias $\leftrightarrow$ Var: Strong independence assumption $\rightarrow$ high bias when features are correlated. Low var (few params). Works surprisingly well despite violated assumptions.

Key HPs: Laplace smoothing α prevents zero probabilities: $\alpha \uparrow \Rightarrow$ more smoothing, higher bias.

LDA (Linear Discriminant Analysis): Gaussian class conditionals with shared $\Sigma \rightarrow$ linear boundary.

$\delta_k(\mathbf{x}) = \mu_k^T \Sigma^{-1} \mathbf{x} - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k$; predict $\arg \max_k \delta_k(\mathbf{x})$.

Estimates: $\hat{\pi}_k = n_k/n$; $\hat{\mu}_k = \frac{1}{n_k} \sum_{i: y_i=k} \mathbf{x}_i$; pooled $\hat{\Sigma}$.

QDA: separate Σ_k per class $\rightarrow$ quadratic boundary; more flexible but more params.

Use when: LDA: Gaussian features, small n , equal covariances, also useful for dim reduction. QDA: Gaussian but unequal covariances.

Bias $\leftrightarrow$ Var: LDA: higher bias (linear boundary) but lower var (fewer params). QDA: lower bias, higher var ($O(p^2)$ params/class). Prefer LDA when n is small relative to p .

Trees & Ensembles

Decision Tree: recursively split on feature/threshold to minimize impurity.

Gini = $1 - \sum_k p_k^2$ (faster); Entropy $H = -\sum_k p_k \log_2 p_k$;

Info gain = $H(\text{parent}) - \sum_c \frac{n_c}{n} H(c)$. Regression:

minimize RSS. Regularize by max depth, min samples/leaf, or pruning (α).

Use when: Tabular data; need interpretability; handles mixed feature types; no scaling needed.

Bias $\leftrightarrow$ Var: Deep tree: very low bias, very high var (memorizes training data). Shallow: high bias, low var.

Pruning trades bias $\uparrow$ for var $\downarrow$.

Key HPs: max_depth ($\uparrow$: var $\downarrow$, bias $\downarrow$); min_samples_leaf ($\uparrow$: var $\downarrow$; ccp-alpha (pruning strength)).

Bagging: train B trees each on a different bootstrap sample; average (regression) or majority vote (classification). Reduces variance; OOB ($\approx 37\%$ of data per tree) gives free validation.

Random Forest: bagging + random subset of $m \approx \sqrt{p}$ (classif.) or $p/3$ (regr.) features at each split.

Decorrelating trees reduces variance further. Feature importance = avg impurity decrease per feature.

Use when: Robust default for tabular, nonlinear data; handles noise and outliers well.

Bias $\leftrightarrow$ Var: Low bias (deep trees) + much lower var than single tree. Adding more trees always helps or is neutral (var $\downarrow$, bias flat).

Key HPs: n_estimators ($\uparrow$: var $\downarrow$, never hurts, just slower); max_features m ($\downarrow$: more decorrelation, slight bias $\uparrow$); max_depth; min_samples_leaf.

AdaBoost: sequential; upweight misclassified examples. Learner weight: $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$. Final:

$H(x) = \text{sign}(\sum_t \alpha_t h_t(x))$.

Gradient Boosting: at each step, fit a new tree to the negative gradient of the loss (pseudo-residuals). Corrects mistakes of the current ensemble.

XGBoost: GBM + L2 leaf regularization + 2nd-order Taylor approximation + column/rf subsampling.

Use when: Tabular data; often beats RF; Kaggle workhorse. Sensitive to noise (unlike RF).

Bias $\leftrightarrow$ Var: Boosting primarily reduces bias (sequential correction). Can overfit with too many rounds $\rightarrow$ use early stopping.

Key HPs: n -estimators / num.boost-rounds ($\uparrow$: bias $\downarrow$, overfit risk $\uparrow$; use early stopping on val set); learning-rate/eta (smaller η needs more rounds but generalizes better; try 0.01–0.1); max_depth (3–6 typical); subsample/colsample.bytree (subsampling, regularizing); reg_lambda ($L2$); min_child_weight.

Neural Networks

Forward pass: $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$; $a^{(l)} = f(z^{(l)})$ (activation applied elementwise).
Activations: ReLU= max(0, x) (default hidden; avoids vanishing gradient); Leaky ReLU (fixes dead neurons: max(αx , x)); sigmoid/softmax at output for probabilities; tanh centered.
Loss: Regression: MSE. Binary classif: BCE. Multiclass: CCE = $-\sum_k y_k \log \hat{p}_k$.

Backpropagation applies the chain rule layer-by-layer to compute gradients:
 $\delta^{(l)} = (W^{(l+1)})^\top \delta^{(l+1)} \odot f'(z^{(l)})$;
 $\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l+1)}(a^{(l)})^\top$

Optimizers: SGD: $\theta \leftarrow \theta - \eta g$. Momentum: $v \leftarrow \beta v - \eta g$, $\theta \leftarrow \theta + v$ (accelerates through ravines). Adam: per-parameter adaptive LR using running estimates of 1st (m) and 2nd (v) moments of gradient; $\theta \leftarrow \theta - \frac{\eta \hat{m}}{\sqrt{\hat{v} + \epsilon}}$. Defaults: $\beta_1=0.9$, $\beta_2=0.999$. Adam usually best default.
Initialization: Xavier $\sim \mathcal{N}(0, \frac{2}{n_{in}+n_{out}})$ for sigmoid/tanh; He $\sim \mathcal{N}(0, \frac{2}{n_{in}})$ for ReLU. Prevents vanishing/exploding gradients at start.

Batch norm: normalize per mini-batch, then scale/shift with learned params; stabilizes, speeds training, mild regularizer.
Use when: Large datasets; images (CNN); sequences (Transformer); when features are unknown/raw.
Bias $\leftrightarrow$ Var: Wide/deep: low bias, high var. Narrow/shallow: high bias. Dropout + weight decay reduce var.
Key HPs: LR η (most critical; use LR schedule or warmup); batch size ($\uparrow$: smoother, but can generalize worse; try 32–256); depth/width ($\uparrow$: capacity $\uparrow$); dropout p (0.1–0.5; var $\downarrow$, bias $\uparrow$); weight decay λ ; epochs + early stopping patience.

CNNs: local filters with shared weights scan across spatial positions $\rightarrow$ learns hierarchical features. Output size = $\lfloor (n + 2p - f)/s \rfloor + 1$. Max pooling adds translation invariance.
Conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow \dots \rightarrow$ FC $\rightarrow$ Softmax.
Use when: Images; any grid-structured data with local patterns (audio, time series).
Key HPs: Filter size f (3 $\times$ 3 common); #filters (capacity); stride s ; padding p .

Transformers: compute self-attention to let every position attend to every other.
 $Q = XW_Q$, $K = XW_K$, $V = XW_V$;
 $A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$. Dividing by $\sqrt{d_k}$ prevents softmax saturation.
Multi-head: h parallel heads capture different relationships. Positional encoding injects order info. $O(n^2d)$ but fully parallelizable.
Use when: Sequences, NLP, long-range dependencies.
Encoder-only (BERT); decoder-only (GPT); encoder-decoder (seq2seq).
Autoencoders: encoder compresses to latent code; decoder reconstructs input. Learns compact representations by minimizing reconstruction error.
Masked autoencoder: randomly mask part of input, encode visible part, reconstruct missing part from context.
Key HPs: #heads h ; model dim d_{model} ; #layers; dropout; LR warmup steps.

PCA & Dimensionality Reduction

Curse of dimensionality: as d grows, data becomes exponentially sparse; distances concentrate (everything looks equally far); KNN and density estimates degrade. Need $n \sim e^d$ to maintain coverage.
PCA finds orthogonal directions of maximum variance (principal components):
(1) Center: $\tilde{X} = X - 1\bar{x}^\top$. (2) SVD $\tilde{X} = U\Sigma V^\top$ or eigen-decomp $S = \frac{1}{n-1}\tilde{X}^\top \tilde{X}$. (3) Sort eigenvectors by eigenvalue $\lambda_k \downarrow$. (4) Project: $Z = \tilde{X}V_{[:,1:d']}$.
Variance explained by PC k : $\lambda_k / \sum_j \lambda_j$. Pick d' via scree

plot elbow or $\geq 90\%$ var retained. Standardize features before PCA if units differ.
Use when: Remove redundant/correlated features; visualization; preprocessing for linear models; noise reduction. Learn PCA transform for training, apply to test.
Bias $\leftrightarrow$ Var: $d' \downarrow$: smoother/denoised (var $\downarrow$, but downstream bias $\uparrow$). $d' \uparrow$: retains more info but keeps noise.
Nonlinear alternatives: t-SNE (visualization only, 2D/3D); UMAP (preserves local + global structure); kernel PCA.
Autoencoder: learn compressed representation by reconstructing input **Masked AE:** learn deeper representations by reconstructing missing parts

Clustering (Unsupervised)

Which method when? K-Means: spherical/equal-size clusters, large n , k known. **GMM:** elliptical clusters, soft membership, also density estimation. **Hierarchical:** unknown k , need dendrogram. **DBSCAN:** arbitrary shapes, outliers, k unknown. **Spectral:** non-convex/manifold clusters; graph connectivity matters more than raw distance.

Distances / similarity: Euclidean (L_2)
= $\sqrt{\sum_j (x_j - y_j)^2}$ for K-Means/hierarchical; Manhattan (L_1) = $\sum_j |x_j - y_j|$ more robust; cosine = $\frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$ for text; Mahalanobis uses covariance when features are correlated. Standardize before Euclidean/Manhattan.

K-Means (hard assignments, minimize within-cluster SSE): initialize centroids $\rightarrow$ assign each point to nearest centroid $\rightarrow$ recompute means $\rightarrow$ repeat. E:
 $C_i = \arg \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$; M: $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$.
Intuition: each centroid is the average representative of its region. Pros: fast/simple. Cons: need k , local minima, sensitive to init/scale, poor for non-spherical clusters. Use K-means++; choose k by elbow or silhouette
 $s(i) = \frac{b(i)-a(i)}{\max(a,b)}$.

Hierarchical (agglomerative): start with each point alone $\rightarrow$ repeatedly merge closest clusters $\rightarrow$ cut dendrogram at chosen height. Linkage: single=min (chains), complete=max (compact), average, Ward=min variance. Intuition: builds a nested family of clusterings. Pros: no k upfront, interpretable tree. Cons: greedy, expensive, early mistakes cannot be undone.

DBSCAN (density-based): choose ϵ , minPts. Mark core points with at least minPts neighbors in ϵ -ball, connect nearby core points, then attach border points; leftovers are noise. Intuition: clusters are dense regions separated by sparse gaps. Pros: arbitrary shapes, finds outliers, no k . Cons: sensitive to ϵ , struggles with varying density/high dimension.

Spectral clustering: build similarity graph $\rightarrow$ graph Laplacian $\rightarrow$ top eigenvectors as embedding $\rightarrow$ run K-Means there. Intuition: cut graph into strongly connected groups, not just nearest points in original space. Pros: handles non-convex clusters. Cons: expensive, must choose similarity graph/ k .

Density estimation asks for smooth $p(x)$, not just cluster labels. **Histogram:** split into bins, estimate density by count/(bin width $\times n$); simple/fast but jagged and bin-width sensitive. **KDE:** place a smooth kernel at each point and sum, $\hat{p}(x) = \frac{1}{nh} \sum_i K\left(\frac{x-x_i}{h}\right)$; steps: choose kernel and bandwidth h , then add all bumps. Intuition: each point spreads local influence. Pros: smooth, nonparametric. Cons: bandwidth-sensitive, slower in high n/d . **GMM:** $\hat{p}(x) = \sum_k \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$ fit by EM: initialize $\rightarrow$ E-step compute responsibilities $r_{ik} \rightarrow$ M-step update $(\pi_k, \mu_k, \Sigma_k) \rightarrow$ repeat. Intuition: data come from several Gaussian sources. Pros: probabilistic, flexible, soft clustering. Cons: need k , init sensitive, Gaussian assumption.

Reinforcement Learning

Framework: agent observes state s , takes action a , gets reward r , transitions to s' . Goal: find policy $\pi(a|s)$ maximizing cumulative reward.
Key components — what each means:
Environment: the world the agent interacts with (chess board, maze, stock market).
State s : complete summary of current situation (board config, robot position, inventory level). Must satisfy Markov property.

Action a : what the agent can do (move piece, go left/right, buy/sell). Set of valid actions A .
Reward r_t : scalar feedback signal received after each action. Encodes the goal (not how to achieve it). Examples: -1 per step in maze (encourages speed); $+1$ on win/0 elsewhere in chess.
Return G_t : total discounted future reward from time t (what we actually optimize).
Policy $\pi(a|s)$: the agent’s strategy — maps states to action probabilities. Deterministic: $a = \pi(s)$. Stochastic: $\pi(a|s) = P(a_t = a | s_t = s)$ (needed for exploration).
State value $V^\pi(s)$: expected return starting from s and following π . “How good is it to be in this state?”
Action value $Q^\pi(s, a)$: expected return from s , taking a once, then following π . “How good is it to take this specific action?”
Scenario mapping tip: identify (1) what decisions are made [actions], (2) what information is available [state], (3) what success/failure looks like [reward], (4) whether task ends [episodic] or continues forever.
Markov property: $P(s_{t+1}|s_t) = P(s_{t+1}|s_t, s_{t-1}, \dots)$ — current state is all you need. **Building blocks:** Markov Chain $\{S, P\} \rightarrow$ MRP $\{R, \gamma\} \rightarrow$ MDP $\{A\}$. RL does NOT assume known P or R (unlike dynamic programming).

Return: $G_t = \sum_{k=0}^\infty \gamma^k R_{t+k+1} = R_{t+1} + \gamma G_{t+1}$.
 $\gamma \rightarrow 0$: myopic (only immediate reward matters); $\gamma \rightarrow 1$: far-sighted (all future matters). Episodic: $G_t = \sum_t R_t$ (no discounting needed but OK). Continuing tasks: need $\gamma < 1$ for convergence.

Value functions (how good is a state or state-action pair):
 $V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$ — expected return from s under π .
 $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$ — expected return from s , taking a , then following π .
Relationship: $V^\pi(s) = \sum_a \pi(a|s)Q^\pi(s, a)$

Bellman equations (recursive decomposition — “backup” property):
 $V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s] = \sum_a \pi(a|s) \sum_{s',r} P(s',r|s,a)[r + \gamma V^\pi(s')]$
 $Q^\pi(s, a) = \sum_{s',r} P(s',r|s,a)[r + \gamma \sum_{a'} \pi(a'|s')Q^\pi(s', a')]$
Bellman optimality:
 $V^*(s) = \max_a \sum_{s',r} P(s',r|s,a)[r + \gamma V^*(s')]$
 $Q^*(s, a) = \max_{s',r} P(s',r|s,a)[r + \gamma \max_{a'} Q^*(s', a')]$

Dynamic Programming (requires full known MDP):
1. Policy Evaluation: iteratively apply Bellman expectation to get V^π from $V_0 = 0$:
 $V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} P[r + \gamma V_k(s')]$. Converges to V^π .
2. Policy Improvement: given V^π , greedily update $\pi'(s) = \arg \max_a Q^\pi(s, a) = \arg \max_a \sum_{s',r} P[r + \gamma V^\pi(s')]$. Guaranteed $V^{\pi'} \geq V^\pi$.
3. Policy Iteration: alternate Eval $\leftrightarrow$ Improve until stable $\rightarrow \pi^*$. (Full eval each time — slow.)
4. Value Iteration: only 1 sweep of eval per improve step. $V_{k+1}(s) = \max_a \sum_{s',r} P[r + \gamma V_k(s')]$. Converges to V^* faster. Extract: $\pi^*(s) = \arg \max_a Q^*(s, a)$.
Key HPs: γ (discount; $\uparrow$ values long-term); convergence threshold θ .

Monte Carlo (model-free; complete episodes; no bootstrapping):
Run episode under π ; average actual returns G_t over all visits to each (s, a) across episodes: $V^\pi(s) \approx \text{avg}(G_t)$. First-visit: count only the first occurrence of s per episode. For control: use $Q^\pi(s, a)$ (no model needed) + ϵ -greedy for exploration.
Bias $\leftrightarrow$ Var: Zero bias (exact returns); high variance (noisy single episodes). Episodic tasks only.
Key HPs: α (step-size); ϵ for ϵ -greedy (decay over time: explore early, exploit later).

Temporal Difference (TD) Learning (model-free; online; bootstraps):
$$V(s) \leftarrow V(s) + \alpha \underbrace{[r + \gamma V(s') - V(s)]}_{\text{TD error } \delta_t}$$

Updates after every step using estimated next-state value $V(s')$ (not waiting for full episode). TD target: $r + \gamma V(s')$.

Bias $\leftrightarrow$ Var: Biased (bootstraps from estimate); lower var than MC; online (doesn’t need episodes to finish).

Q-Learning (off-policy TD):
 $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
Uses greedy max over next state regardless of action taken (off-policy). Converges to Q^* .
SARSA (on-policy):
 $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$, $a' \sim \pi$.
Safer; learns value of policy it actually follows (avoids risky shortcuts).
Use when: Q-learning: find optimal policy offline. SARSA: safe online learning where bad behavior has real cost.
Key HPs: α (LR; decay for convergence); ϵ decay schedule; γ .

Exploration strategies: ϵ -greedy: prob ϵ take random action; else $\arg \max_a Q$. Decay ϵ over time. UCB: $a^* = \arg \max_a [Q(a) + c\sqrt{\ln t/N(a)}]$ (optimism in the face of uncertainty; $c \uparrow$: more exploration).
Policy Gradient (REINFORCE):
 $\nabla_\theta J(\theta) = \mathbb{E}[G_t \nabla_\theta \log \pi_\theta(a_t | s_t)]$. Works with continuous action spaces; high variance; subtract a baseline (e.g. $V^\pi(s)$) to reduce variance.
DQN: uses a neural network to approximate $Q^*(s, a)$ for large/continuous state spaces. Two tricks for stability: *experience replay* (sample past transitions randomly to break correlation) and *target network* (separate frozen copy of Q network updated periodically).

Gradient Descent & Optimization

$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$. LR η too high $\rightarrow$ diverge; too low $\rightarrow$ very slow.
Batch GD: uses all data each step (stable, slow). **SGD:** one sample (fast, noisy, can escape local minima).
Mini-batch (32–256): standard in practice (balances stability and speed).
LR schedules: step decay, cosine annealing, warmup + decay. Adam is usually the best default.
Convex loss $\Rightarrow$ GD finds global min. NN losses are non-convex; saddle points are more common than local minima in high- d ; SGD noise helps escape them.

Practical Reference

Feature scaling: standardize ($\mu=0$, $\sigma=1$) for KNN, PCA, NN, GD-based, regularized models. *Not* needed for tree-based methods.
Overfit (train $\ll$ val error): add regularization, get more data, use simpler model, dropout, early stopping, data augmentation.
Underfit (both errors high): increase capacity (depth/width), add features, reduce regularization, train longer.
Loss map: Regression: MSE (outlier-sensitive)/MAE (robust)/Huber. Binary: BCE. Multiclass: CCE.
CV discipline: all preprocessing *inside* the CV loop; never touch the test set until final evaluation; tune on val, report on test.
Collinearity: VIF >10 is concerning; Ridge handles better than LASSO (LASSO may pick one correlated feature arbitrarily). Large $p/n \Rightarrow$ LASSO or elastic net.
Class imbalance: SMOTE, class weights, PR-AUC over ROC-AUC, threshold from cost matrix.

Full model comparison:
Lin/Logistic best: linear, interpretable effects; pros: fast, simple, calibrated probs/coeffs; cons: misses nonlinearities unless engineered.
KNN best: small nonlinear problems; pros: nonparametric, intuitive; cons: slow at prediction, scaling-sensitive, weak in high d .
NB/LDA/QDA best: small- n probabilistic baselines; pros: fast, low variance, good if assumptions fit; cons: assumptions may fail, QDA higher variance.
Trees/RF best: tabular nonlinear data; pros: little preprocessing, automatic interactions, robust; cons: trees overfit, RF less interpretable.
Boost/XGB best: strong tabular performance; pros: high accuracy, flexible; cons: more tuning, noise/overfit risk.
NN/CNN best: large unstructured data; pros: learns features, very flexible; cons: data/compute hungry, low interpretability.
Transformer best: text/sequence context; pros: parallel, captures long-range deps; cons: attention cost high, large-data hungry.